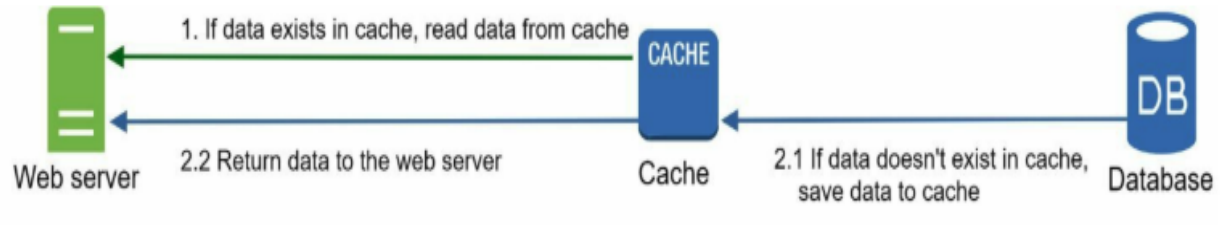
CACHE

Cache is a temporary storage layer that keeps copies of frequently accessed data closer to where it's needed, so future requests for that data can be served faster instead of recomputing or refetching it. Caches can exist at many levels — browser, application, database, or server — and are typically fast but limited in size, often using strategies like LRU (Least Recently Used) to decide what to keep or evict.



KEY CONCEPTS

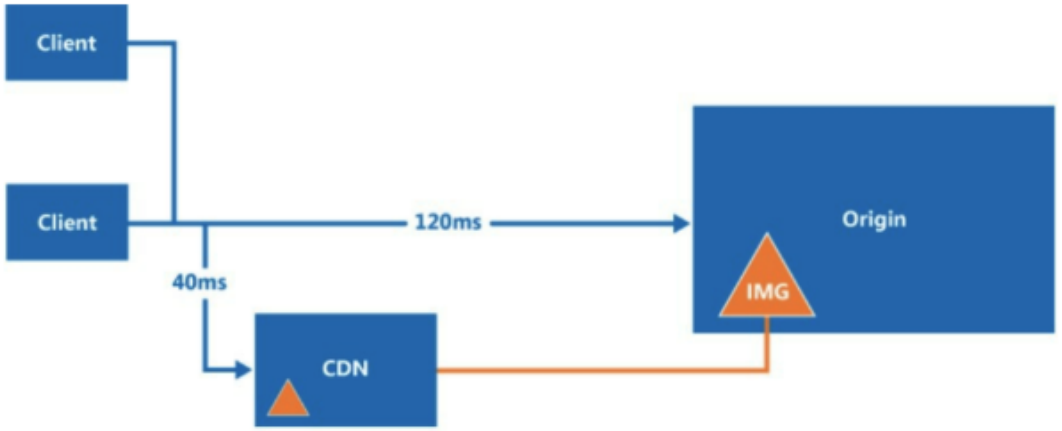
- Purpose: reduce latency and backend load
- TTL (expiration) - Time to live
- Eviction policies: LRU, LFU, FIFO
- Cache invalidation
- Write-through
- Write-back (write-behind)
- Write-around
- Levels of caching
    - client-side, CDN,
    - Application, Distributed,
    - DB query cache
- Consistency trade-off (staleness vs. performance)

TYPES OF CACHE

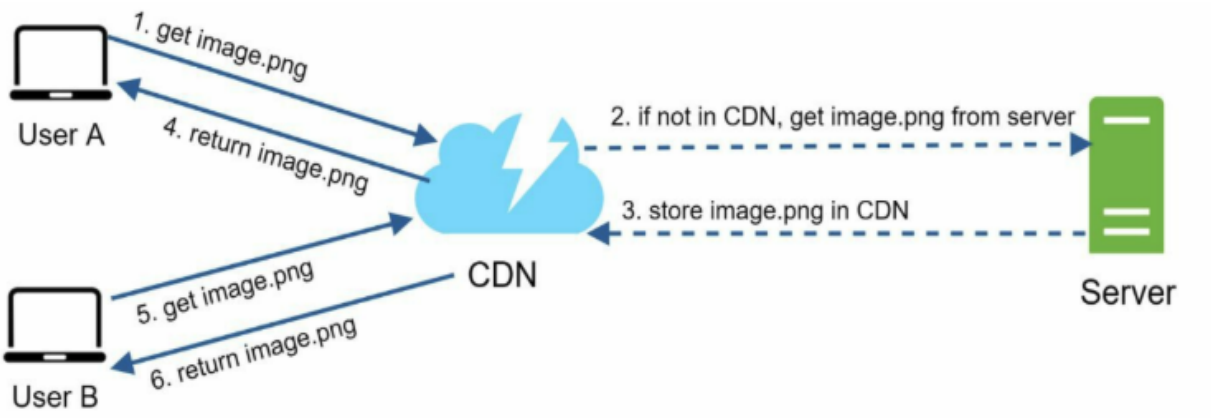
- In memory cache - Redis, Memcache, Caffeine cache
- Distributed cache - Redis (Cluster mode), Hazelcast, AWS Elastic cache

CONTENT DELIVERY NETWORK

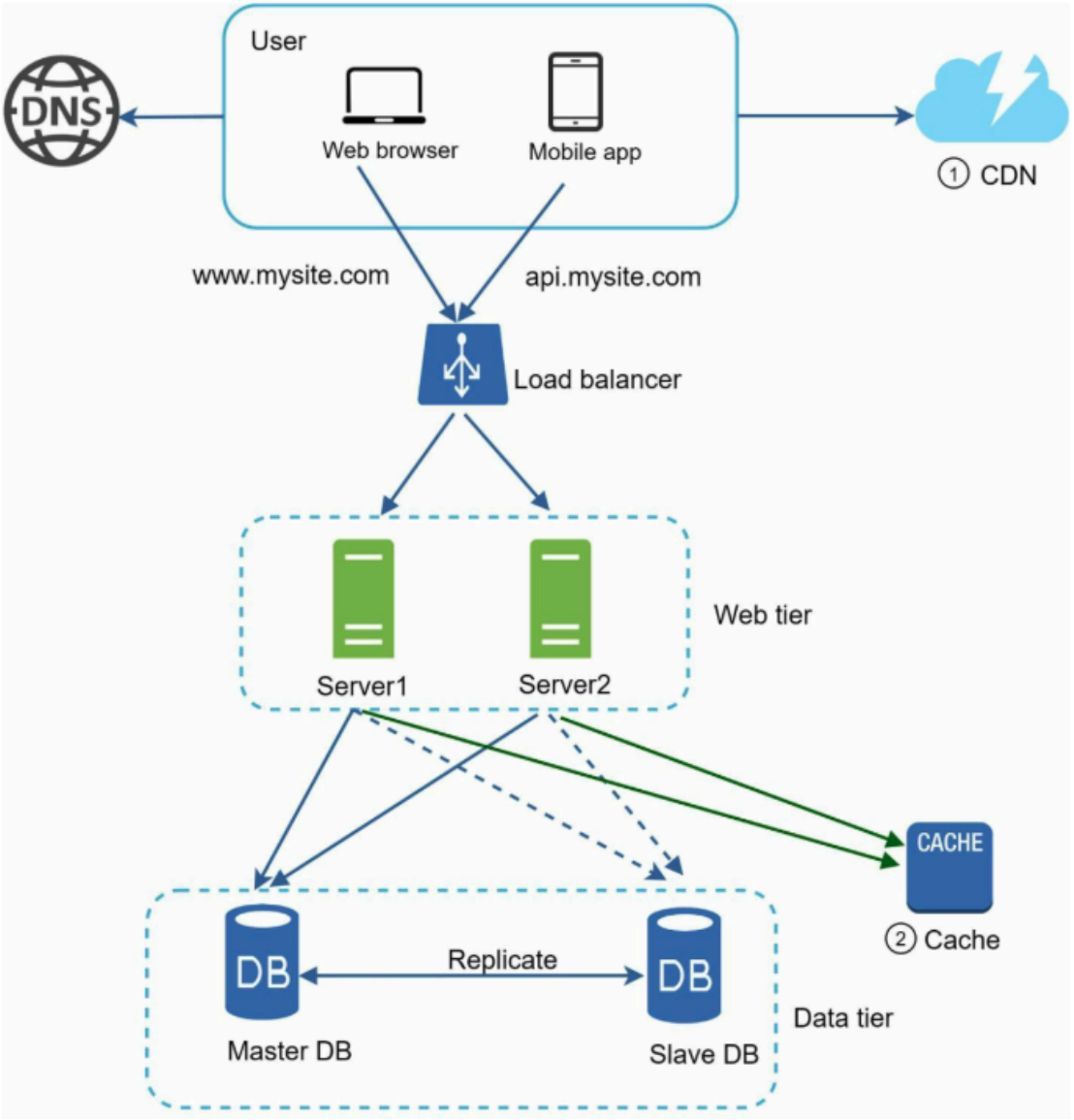
CDN (Content Delivery Network) is a geographically distributed network of servers that cache and deliver content (like images, videos, scripts, and web pages) from a location physically closer to the end user. This reduces latency, decreases load on the origin server, and improves overall website performance and reliability, especially for users far from where the content originally lives.



- Purpose: distribute content via “edge servers” to reduce latency
- Edge servers (PoPs)
- Origin server
- Static vs. dynamic content
- Reduced origin load
- Latency reduction
- DDoS mitigation & security

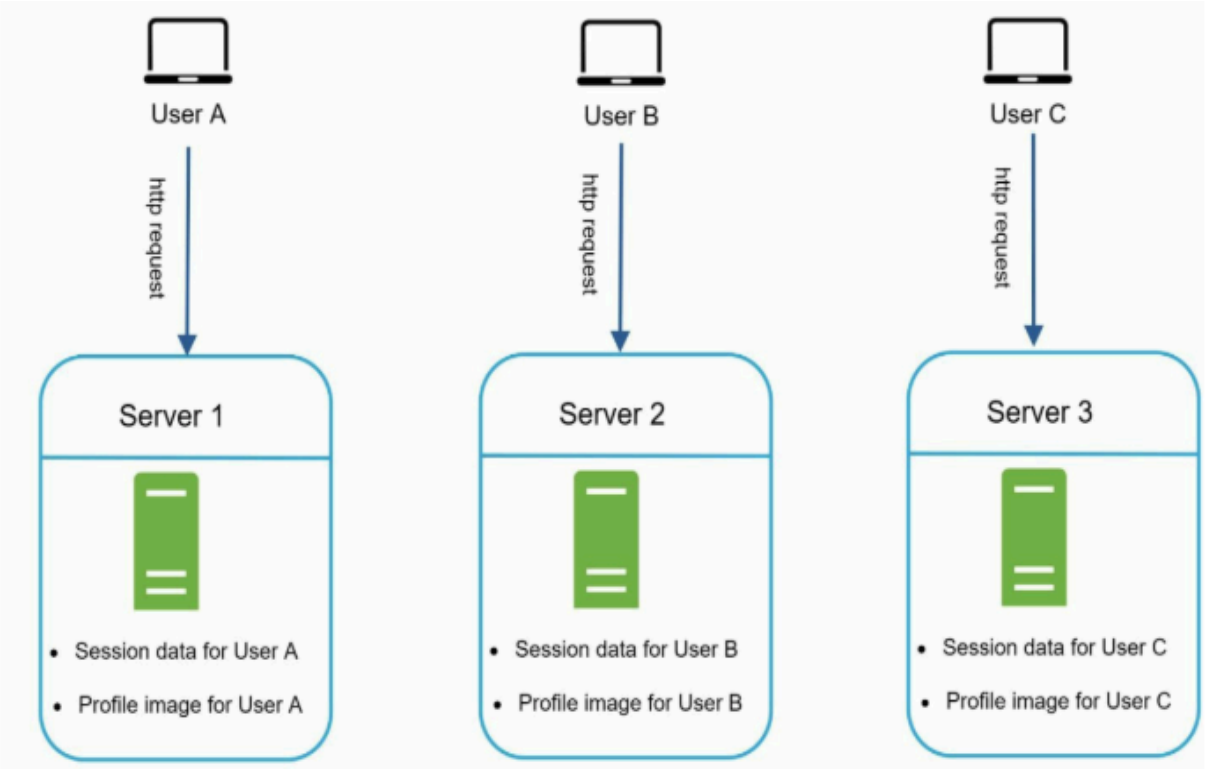


ARCHITECTURE



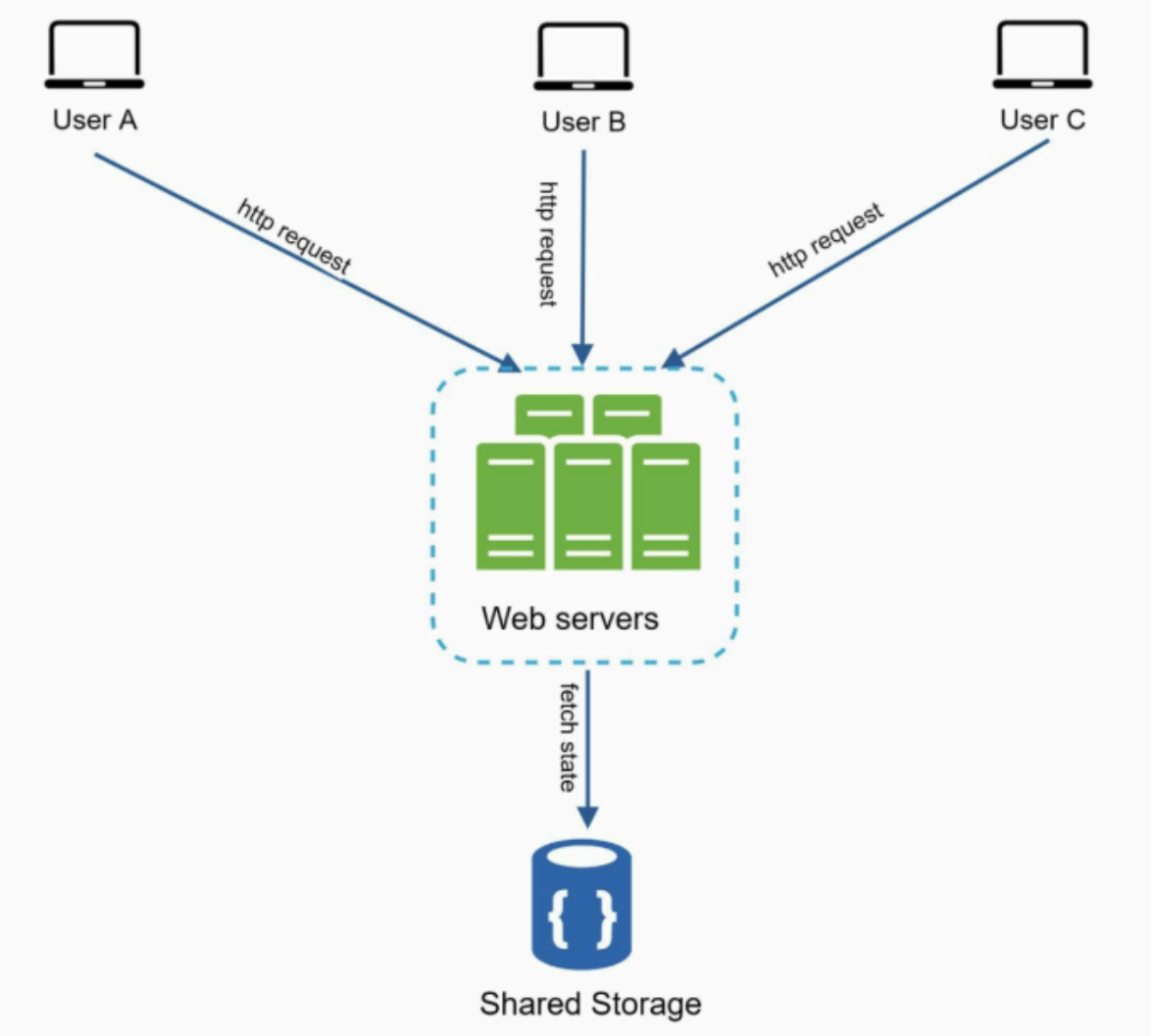
STATEFUL VS STATELESS ARCHITECTURE

STATEFUL



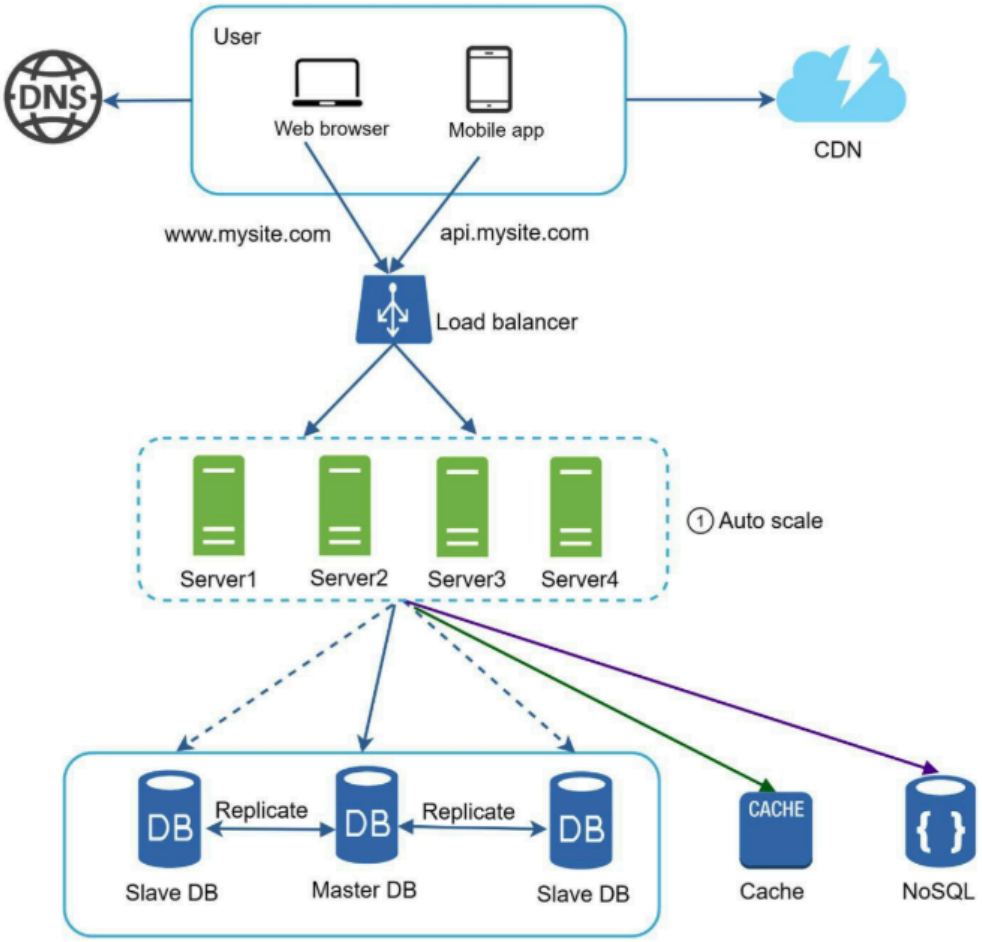
1. Harder to horizontally scale
2. Requires sticky sessions / session replication
3. Load balancing is less flexible
4. Single point of failure
5. Poor fault tolerance / failover
6. Resource inefficiency (idle memory reserved per client)
7. Complex deployments (risk of dropping sessions)
8. Not container/cloud-native friendly
9. Synchronization overhead across servers
10. Harder to test and debug

STATELESS ARCHITECTURE

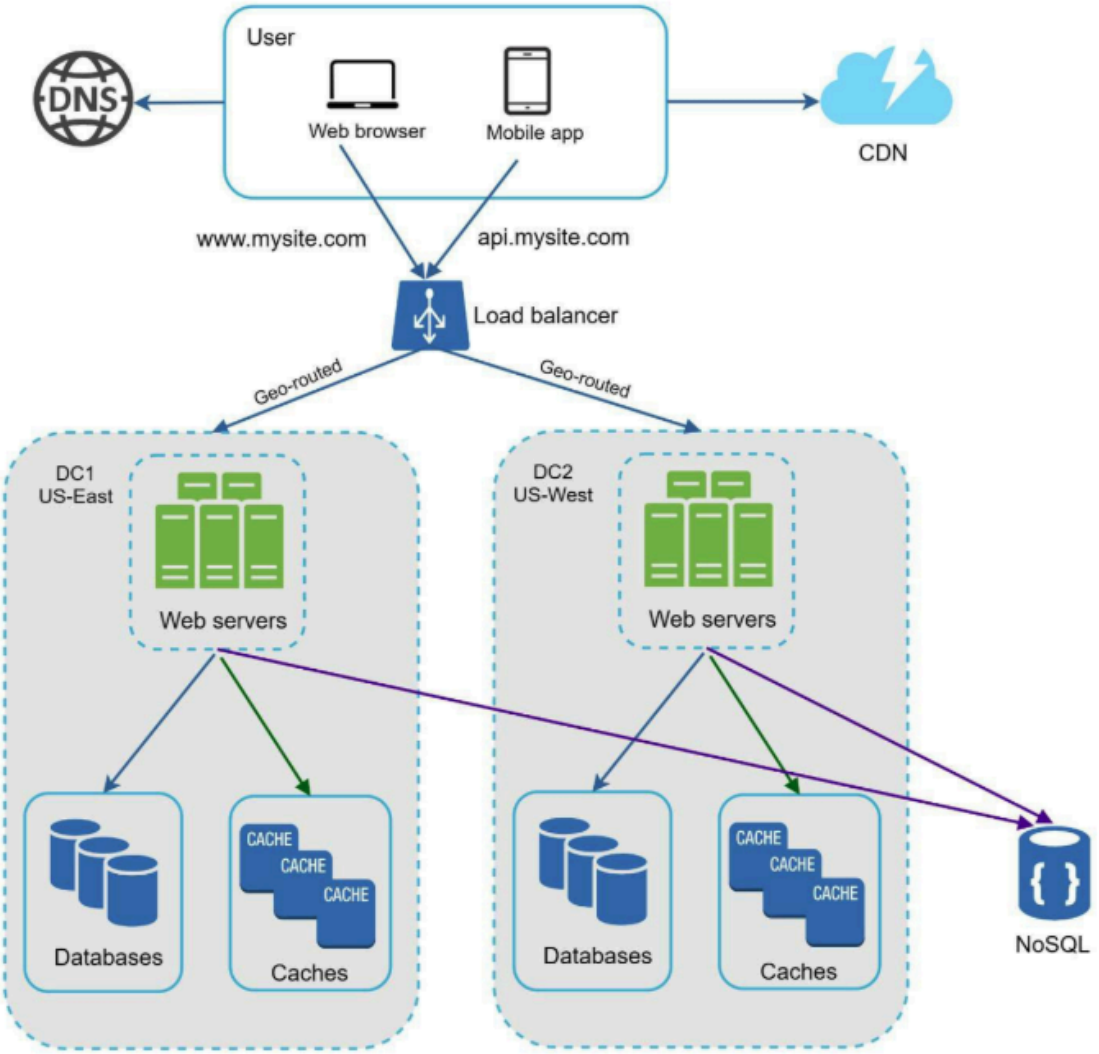


1. Easy horizontal scaling
2. No sticky sessions needed
3. Flexible load balancing
4. No single point of failure
5. Better fault tolerance / easy failover
6. Efficient resource usage
7. Simple deployments (no session loss risk)
8. Cloud-native / container friendly
9. No state synchronization needed

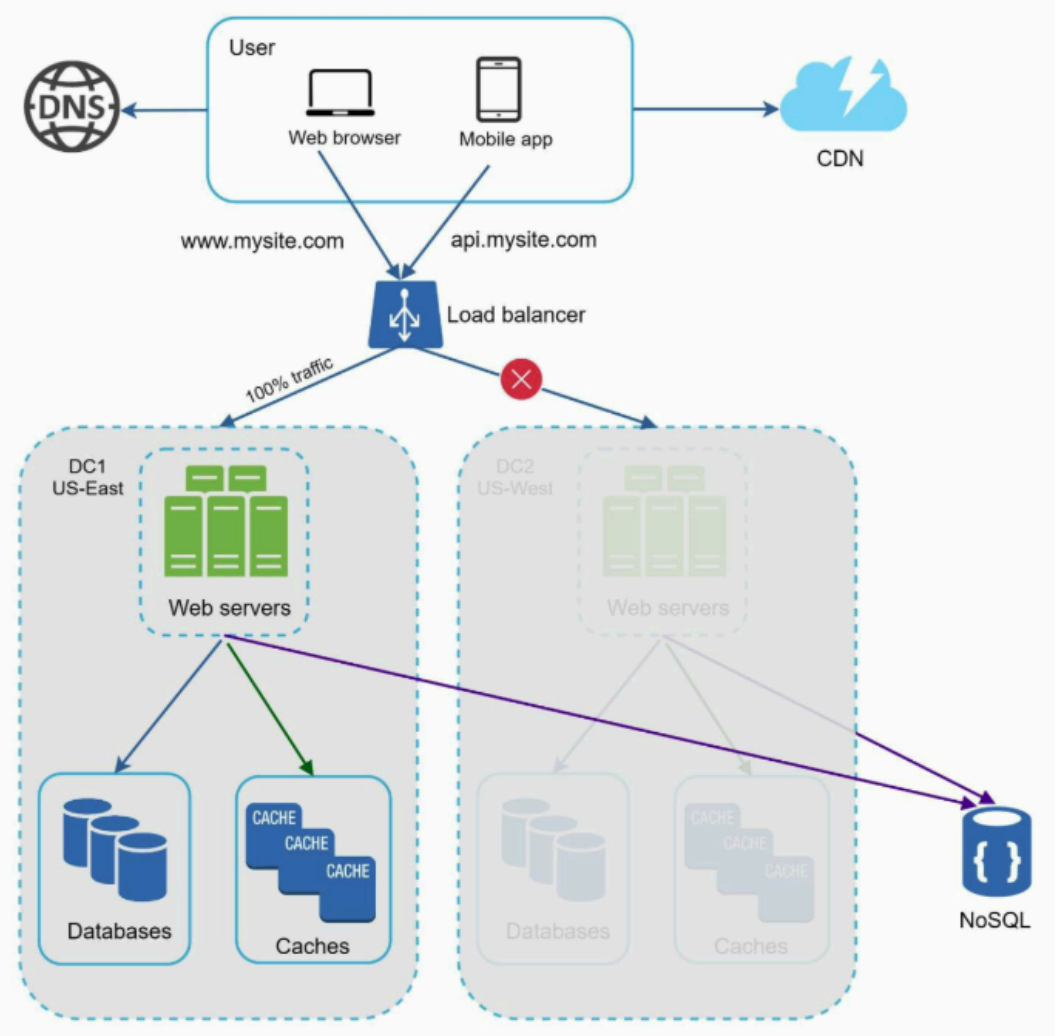
ARCHITECTURE



DATA CENTERS



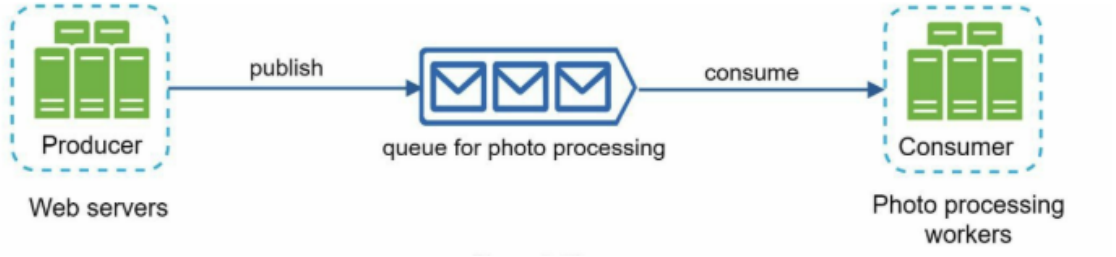
DISASTER RECOVERY



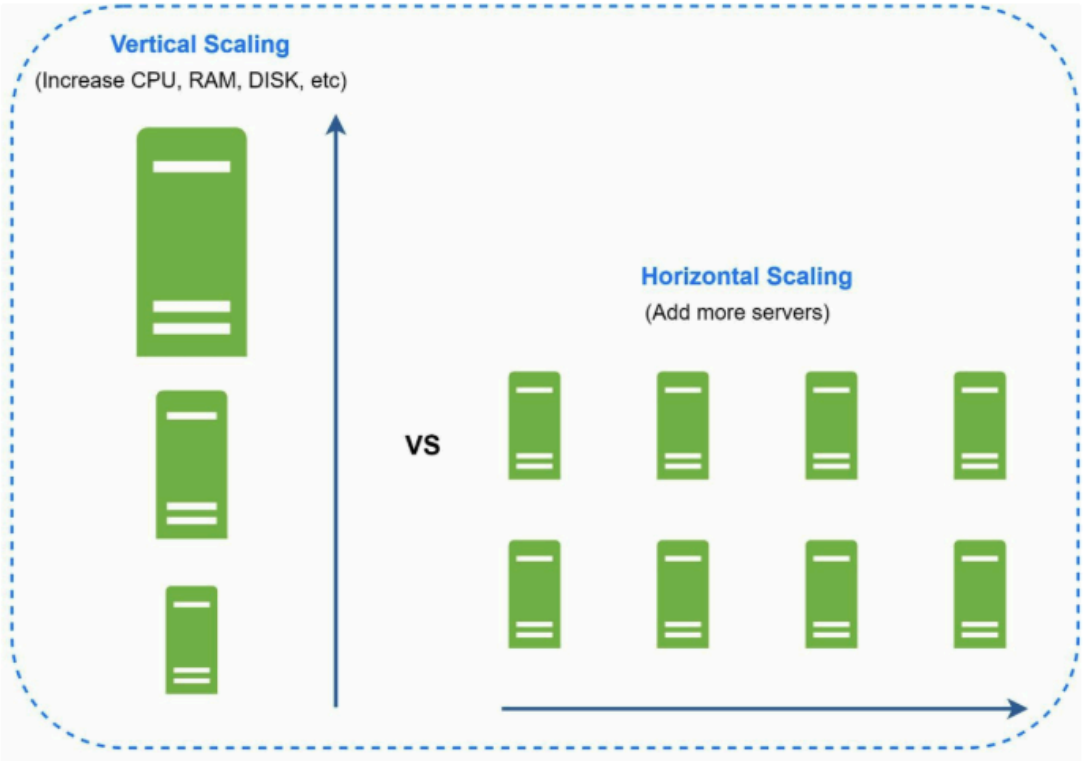
Data center → Group of availability zones (Minimum 2, Maximum 3) → exact hardware warehouse

MESSAGE QUEUE

Photo Processing example



DATABASE SCALING

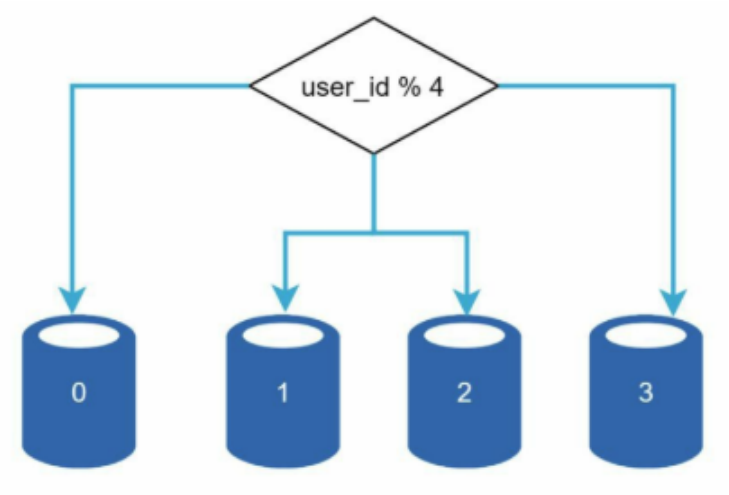


SHARDING

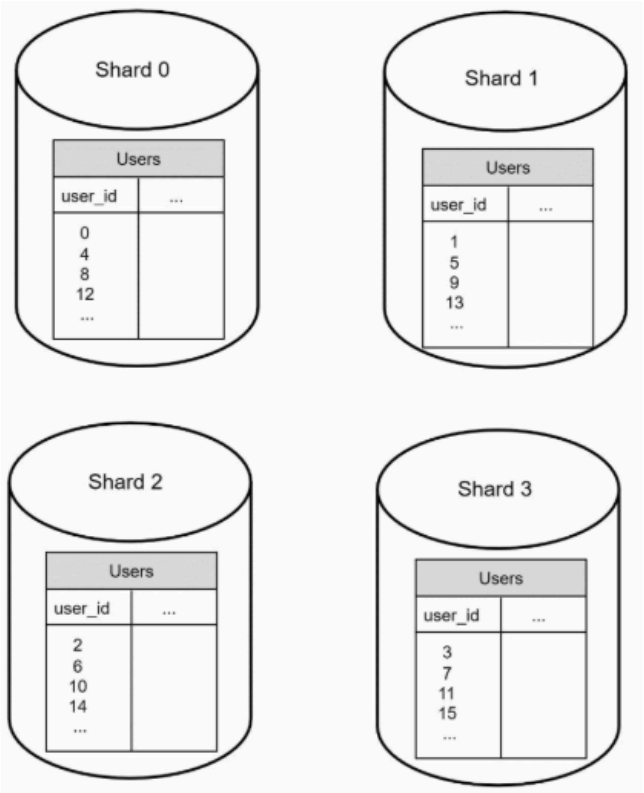
- Horizontal scaling, also known as sharding, is the practice of adding more servers.
- Sharding separates large databases into smaller, more easily managed parts called shards.
- Each shard shares the same schema, though the actual data on each shard is unique to the shard.

HOW DATA IS MANAGED?

- User data is allocated to a database server based on user IDs. Anytime you access data, a hash function is used to find the corresponding shard.
- For example, user_id % 4 is used as the hash function. If the result equals to 0, shard 0 is used to store and fetch data. If the result equals 1, shard 1 is used. The same logic applies to other shards.



Below shows the user table in sharded databases.



FINAL ARCHITECTURE